Options for Queue Based Enrichment System

My thoughts on the requirements to discuss:

- Expected number of enrichments per minute/hour
  - Likely up to 100 per minute, like if we're running 4 enrichments across 15 docs
- Ability to process enrichments simultaneously
  - I think this is necessary because it lets the system run at full capacity w/o worrying about enrichments
- One enrichment type per enrichment request
  - If multiple per request, is sequential processing necessary?
  - I think we're only dealing with one per request right now, but that could change in the future.
- Criticality of retry mechanisms, low
  - We just need to store that a job failed as we discussed Friday.
- Audit and job history trails are necessary
  - Make sure we know what happens with every job
- Persistence is not necessary across service restarts
  - We just need to know that the last job of the prior service's life failed
- Job processing delays are not too big of a deal
  - This is a backend service behind a backend service, users can wait to access the embeddings of their docs
- Are enrichments CPU bound? I/O bound?
  - Embeddings: I/O bound, has to wait for OpenAI
  - Date extraction: CPU-bound due to text parsing, pattern recognition
  - Glossary: mostly I/O due to Neo operations
  - Summary: I/O bound, uses OpenAI
- What are the memory constraints of the system?
- OpenAI gives external API rate limits
  - This slows us down some
- Job priority is a necessity
  - Do enrichments first because they are more important
- Is job scheduling necessary? I think not.
  - Jobs can be run ASAP, esp if push based queue system
- Do we need to be able to cancel jobs? I think not.
  - Have never spoken about this, seems unnecessary
- Real-time status monitoring is necessary
  - Need to have real time status of jobs
- We need metrics for things like queue depth, processing times, and failure rates
  - More data is more good

Option 1: Current System

Pros	Cons
Simple, currently functional	No persistence
Familiar codebase	Partial job history
No new dependencies	Limited monitoring
Low memory footprint	Single threaded
	No rate limiting

Option 2: Piscena as Queue and Worker Threads

Pros	Cons
True parallelism	No persistence
Simple architecture	No native job history or audit trail
Good for CPU bound work	No built-in monitoring
Low latency	No retry mechanism
	No priority handling
	No rate limiting

Option 3: BullMQ without Worker Threads

Pros	Cons
Full persistence	Single threaded
Excellent job history, monitoring, and auditing	Redis dependency
Built in rate limiting	Slower for CPU tasks
Native support for priority queues	

Option 4: BullMQ + Piscena Worker Threads

Pros	Cons
All BullMQ pros	More complex
Parallelism	Higher memory usage
Best performance	More dependencies
Flexible resource allocation	Debugging complexity